

Deep Learning Basics

1. Neural Network Architecture

Neural networks consist of neurons, layers, and weights/biases. Neurons process inputs through layers, where weights and biases adjust the learning process. Common network types include:

- **Feedforward Networks:** Basic structure where data flows in one direction.
- **Convolutional Networks:** Primarily used for image data to detect patterns.
- **Recurrent Networks:** Ideal for sequential data (e.g., text, time series).

2. Forward and Backward Propagation

- **Forward Propagation:** Data is processed through layers to generate output.
- **Backpropagation:** Uses the chain rule and gradient descent to update weights and biases, minimizing the error by adjusting learning with a defined learning rate.

3. Activation Functions

Common functions include:

- **ReLU:** Rectified Linear Unit for non-linearity.
- **Sigmoid:** Outputs values between 0 and 1.
- **Tanh:** Hyperbolic tangent function, outputs values between -1 and 1. Advanced functions like **Leaky ReLU**, **ELU**, and **SELU** handle specific challenges like dead neurons and vanishing gradients.

4. Deep Learning Frameworks - TensorFlow/Keras

- **TensorFlow:** Operates with tensors (data structures), variables, and operations.
- **Keras:** Simplifies model-building:
 - **Sequential API:** Linear stack of layers.
 - **Functional API:** For more flexible, complex models.
 - **Model Subclassing:** Custom network architectures.

Keras also offers tools for configuring layers, loss functions, and optimizers.

5. PyTorch Basics

- **PyTorch:** Utilizes tensors and autograd for automatic differentiation.
- **Model Development:** Includes:
 - **Dataset/DataLoader** for efficient data handling.
 - **Training Loops:** Customizable for specific needs.
 - **Model Evaluation:** Tools for evaluating and optimizing models.

Importing Required Libraries

1. Numpy and Pandas

- **numpy**: A library for numerical computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays.
- **pandas**: A library used for data manipulation and analysis. It provides data structures like DataFrames, which are useful for handling and analyzing structured data (e.g., CSV files, Excel spreadsheets).

2. Matplotlib and Seaborn

- **matplotlib.pyplot**: A plotting library used to create static, animated, and interactive visualizations in Python. Here, it's used for creating various charts and graphs.
- **seaborn**: A data visualization library based on Matplotlib. It provides a high-level interface for creating attractive and informative statistical graphics.

3. Scikit-learn

- **sklearn.datasets** : A module to load and generate datasets. This code imports functions to generate synthetic datasets for classification and regression tasks and load predefined datasets like digits.
 - **make_classification** : Generates a synthetic dataset for classification tasks.
 - **make_regression** : Generates a synthetic dataset for regression tasks.
 - **load_digits** : Loads the digits dataset, which is a multi-class classification dataset containing images of digits.
- **sklearn.model_selection** : Provides functions for splitting datasets into training and testing sets, such as **train_test_split**, which is used here to split data.
- **sklearn.preprocessing** : Includes methods for data preprocessing like **StandardScaler**, which standardizes data (scaling features to have mean=0 and variance=1).

4. TensorFlow/Keras

- **tensorflow** : An open-source deep learning framework developed by Google. It allows for the creation and training of neural networks and other machine learning models.
- **keras** : A high-level neural network API running on top of TensorFlow. It simplifies the process of creating and training deep learning models.

5. PyTorch

- **torch** : A deep learning framework developed by Facebook, which provides a flexible and easy-to-use way to build machine learning models.
- **torch.nn** : A submodule of PyTorch containing classes and methods to build neural networks.
- **torch.optim** : Provides optimization algorithms (e.g., Adam, SGD) to update model parameters during training.
- **torch.utils.data.Dataset** and **DataLoader** : Utilities to handle datasets in PyTorch. **Dataset** is an abstract class for datasets, and **DataLoader** is used for batch processing during model training.

6. Warnings

- **warnings.filterwarnings("ignore")** : Suppresses warning messages to keep the output clean. In this case, it suppresses warnings that might clutter the notebook.

7. Matplotlib Inline Magic

- **%matplotlib inline** : This magic command ensures that plots are rendered directly in the notebook rather than in a separate window. It is commented out here, but when used, it makes visualizations appear within the notebook.

8. Setting Plot Style

- **plt.style.use('seaborn')** : This sets the default plotting style to Seaborn's style for cleaner, more aesthetically pleasing charts.

```
In [62]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import make_classification, make_regression, load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow import keras
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import warnings
warnings.filterwarnings("ignore")
# %matplotlib inline
plt.style.use('seaborn')
```

1. Neural Network Components

1. Neurons

- **What it is:** Neurons are the basic units of a neural network, inspired by the brain. Each

neuron receives inputs, processes them, and passes the output to the next layer. Neurons simulate the way the human brain works to recognize patterns and make predictions.

2. Layers

- **What it is:** Layers are collections of neurons. Neural networks are made up of different layers:
 - **Input Layer:** Receives the input data.
 - **Hidden Layers:** Perform computations and extract features from the input data.
 - **Output Layer:** Produces the final result or prediction.

3. Weights and Biases

- **What it is:**
 - **Weights:** The parameters that adjust the input data in each neuron, determining how much influence an input will have on the output.
 - **Biases:** Additional parameters added to the input to shift the output, allowing the model to better fit the data.

Network Types

1. Feedforward Networks

- **What it is:** A simple type of neural network where the information flows in one direction, from input to output. It has no loops or cycles, and data is passed forward through the network.

2. Convolutional Networks (CNNs)

- **What it is:** A specialized neural network for processing grid-like data, such as images. It uses **convolutional layers** to detect patterns like edges, shapes, and textures in images, making it ideal for image classification.

3. Recurrent Networks (RNNs)

- **What it is:** A type of neural network where the information loops back on itself, allowing the network to use previous outputs as part of its input. This makes it great for tasks like time-series forecasting and natural language processing, where context is important.

Forward/Backward Propagation

Forward Propagation

1. Input Processing

- **What it is:** The first step in forward propagation where the raw data is fed into the neural network. This data can be anything from numbers to images.

2. Layer Computation

- **What it is:** The data passes through different layers of the network, where each layer performs mathematical operations on the data to extract useful features.

3. Output Generation

- **What it is:** The final output is generated after the data passes through all the layers. This output could be a predicted class, continuous value, or decision based on the problem being solved.

Backpropagation

1. Chain Rule

- **What it is:** A mathematical principle used in backpropagation to calculate the gradient (rate of change) of the loss function with respect to each weight and bias. It allows the model to update its parameters to improve performance.

2. Gradient Descent

- **What it is:** A technique used to minimize the error (or loss) by updating the model's weights. It involves calculating the gradient and adjusting the weights in the direction that reduces the error.

3. Learning Rate

- **What it is:** A hyperparameter that controls how large a step the model takes when updating the weights during gradient descent. A small learning rate may slow down training, while a large learning rate may cause the model to miss the optimal solution.

Neural Networks are computational models inspired by the structure and function of biological neural networks in the brain. They are widely used in machine learning for tasks like classification, regression, and more.

Key Components of a Neural Network:

1. Neurons (Nodes):

- Neurons are the basic units of a neural network. Each neuron receives input, processes it, and produces output. The output is then passed on to the next layer of neurons.

2. Layers:

- **Input Layer:** The first layer that receives the raw data (features).
- **Hidden Layers:** Intermediate layers that process the information and learn patterns.
- **Output Layer:** The final layer that produces the model's predictions or decisions.

3. Weights and Biases:

- **Weights:** These control the strength of the connection between neurons. Each connection has a weight that is adjusted during training.
- **Biases:** These shift the activation function and help the model fit better by adding flexibility.

4. Activation Functions:

- Activation functions are mathematical operations applied to the output of each neuron. They introduce non-linearity, allowing the network to learn complex patterns.

Visualizing a Simple Neural Network Architecture

In the code below, a simple neural network architecture is visualized. This network consists of:

- **Input Layer** with 4 neurons
- **Hidden Layer 1** with 5 neurons
- **Hidden Layer 2** with 3 neurons
- **Output Layer** with 2 neurons

The neurons in each layer are represented by blue dots, and the connections between the layers are shown as gray lines.

The visualization helps understand how data flows through the network: from the input layer through the hidden layers and finally to the output layer.

```
In [63]: # Cell 1: Introduction to Neural Networks
        """
        Neural Networks are computational models inspired by biological neural networks.
        Key components:
        1. Neurons (nodes)
        2. Layers (input, hidden, output)
        3. Weights and biases
        4. Activation functions
        """

        # Visualize a simple neural network architecture
        def plot_neural_network():
            fig, ax = plt.subplots(figsize=(10, 6))

            # Layers
            layer_sizes = [4, 5, 3, 2]
            layer_positions = [1, 2, 3, 4]

            # Plot neurons
            for i, size in enumerate(layer_sizes):
                y_positions = np.linspace(0, size-1, size)
                for y in y_positions:
                    ax.scatter(layer_positions[i], y, c='blue', s=100)

            # Add layer labels
            if i == 0:
                ax.text(layer_positions[i]-0.1, size/2, 'Input\nLayer', ha='right')
            elif i == len(layer_sizes)-1:
                ax.text(layer_positions[i]+0.1, size/2, 'Output\nLayer', ha='left')
            else:
                ax.text(layer_positions[i], size+0.5, f'Hidden\nLayer {i}', ha='center')

            # Plot connections
            for i in range(len(layer_sizes)-1):
```

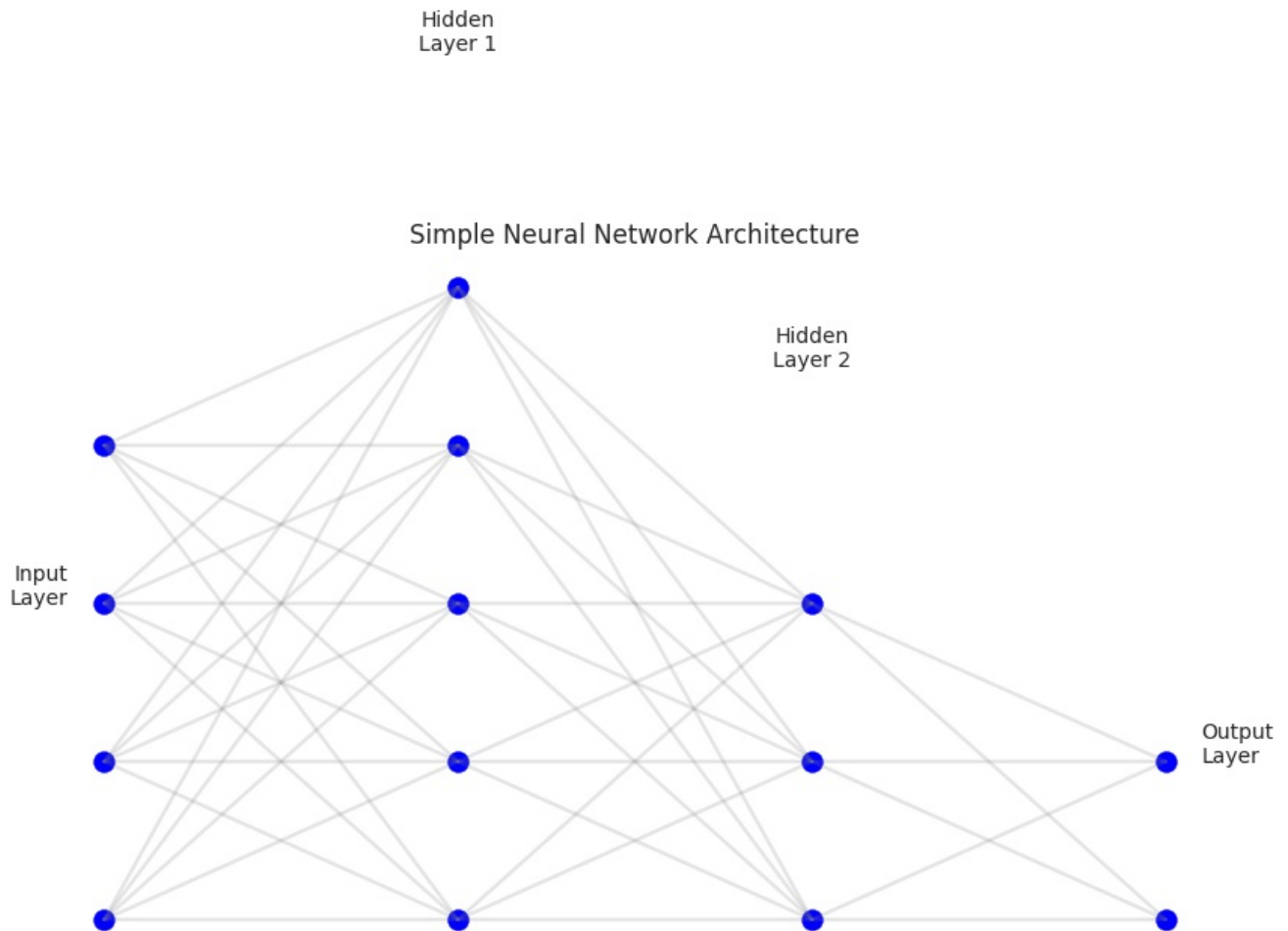
```

for j in range(layer_sizes[i]):
    for k in range(layer_sizes[i+1]):
        ax.plot([layer_positions[i], layer_positions[i+1]],
                [j, k], 'gray', alpha=0.2)

ax.set_title('Simple Neural Network Architecture')
ax.axis('off')
plt.show()

plot_neural_network()

```



2. Activation Functions

1. Common Functions

- **ReLU (Rectified Linear Unit)**

- **What it is:** A popular activation function that outputs the input if it's positive, and zero otherwise. It introduces non-linearity to the model, making it capable of solving more complex problems.

- **Sigmoid**

- **What it is:** A function that outputs values between 0 and 1. It's often used for binary classification problems where the output needs to represent a probability.

- **Tanh (Hyperbolic Tangent)**

- **What it is:** Similar to the sigmoid function but outputs values between -1 and 1. It is often used when the model needs to output values in a balanced range.

2. Advanced Functions

- **Leaky ReLU**

- **What it is:** A variant of ReLU that allows a small negative slope for negative input values, which helps solve the problem of dead neurons (where ReLU outputs zero for all inputs).

- **ELU (Exponential Linear Unit)**

- **What it is:** An activation function that outputs a smooth, continuous curve for both positive and negative inputs, helping improve model training speed and performance.

- **SELU (Scaled Exponential Linear Unit)**

- **What it is:** An advanced version of ELU that automatically scales the activations during training, often improving training stability and speed.

1. Purpose of Activation Functions

- Activation functions are crucial in neural networks because they introduce non-linearity into the model. Without them, the network would only be able to learn linear relationships, which is limiting. The non-linearity allows the model to learn and represent more complex patterns in the data.

2. Types of Activation Functions in the Code:

- **ReLU (Rectified Linear Unit):**

- **Formula:** $f(x) = \max(0, x)$
- **Behavior:** It outputs the input directly if it is positive; otherwise, it outputs zero. This helps with the vanishing gradient problem and is commonly used in hidden layers.

- **Sigmoid:**

- **Formula:** $f(x) = 1 / (1 + \exp(-x))$
- **Behavior:** It squashes the output to a range between 0 and 1, which is useful for binary classification problems (e.g., outputting probabilities).

- **Tanh (Hyperbolic Tangent):**

- **Formula:** $f(x) = \tanh(x)$
- **Behavior:** Similar to sigmoid but squashes the output to a range between -1 and 1, which helps with zero-centered outputs.

- **Leaky ReLU:**

- **Formula:** $f(x) = x$ if $x > 0$ else $0.1 * x$
- **Behavior:** Similar to ReLU, but for negative values of x , it allows a small negative slope rather than outputting zero. This helps to avoid "dead neurons" that might occur in standard ReLU.

3. Function to Plot Activation Functions:

- **Plotting:** The function `plot_activation_functions()` plots the behavior of each activation function over a range of input values from -5 to 5.
- **What it does:**
 - Generates a range of values (x) from -5 to 5.
 - Computes the output of each activation function for every value in x .
 - Plots the output for each function in a 2x2 grid of subplots.

4. Explanation of the Plots:

- **ReLU Plot:** The output is zero for negative inputs and linear for positive inputs.
- **Sigmoid Plot:** The output smoothly curves between 0 and 1.
- **Tanh Plot:** The output smoothly curves between -1 and 1.
- **Leaky ReLU Plot:** Similar to ReLU, but allows a small negative slope for negative inputs.

5. Why Plot These Functions?

- Visualizing these functions helps to understand how each one shapes the input data and how they contribute to a neural network's ability to learn complex patterns.

```
In [64]: # Cell 2: Activation Functions
        """
        Activation functions introduce non-linearity into the network,
        allowing it to learn complex patterns.
        """

        def plot_activation_functions():
            x = np.linspace(-5, 5, 100)

            # ReLU
            relu = np.maximum(0, x)

            # Sigmoid
            sigmoid = 1 / (1 + np.exp(-x))

            # Tanh
            tanh = np.tanh(x)

            # Leaky ReLU
```

```
leaky_relu = np.where(x > 0, x, 0.1 * x)

fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(12, 8))

ax1.plot(x, relu)
ax1.set_title('ReLU')
ax1.grid(True)

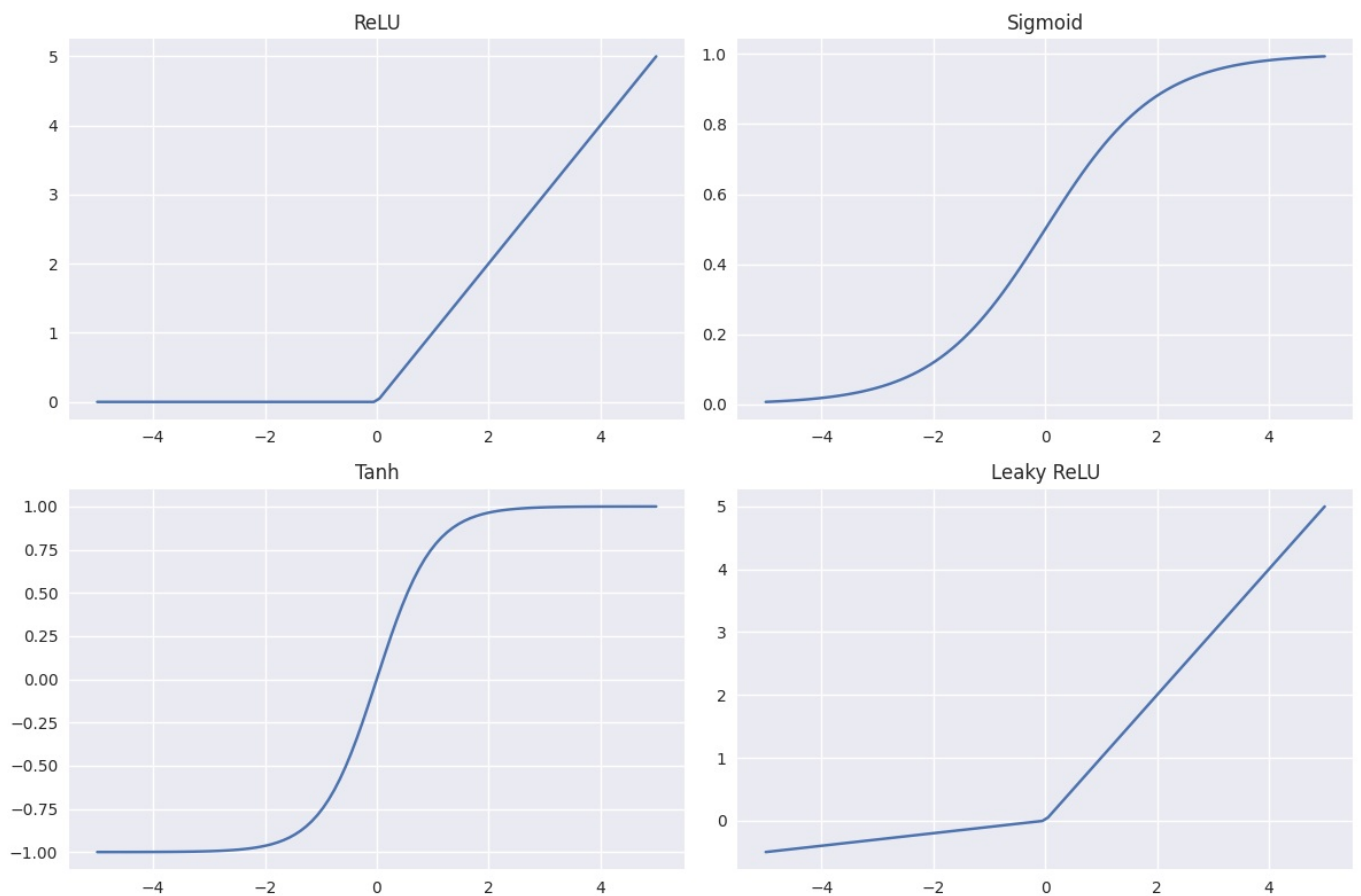
ax2.plot(x, sigmoid)
ax2.set_title('Sigmoid')
ax2.grid(True)

ax3.plot(x, tanh)
ax3.set_title('Tanh')
ax3.grid(True)

ax4.plot(x, leaky_relu)
ax4.set_title('Leaky ReLU')
ax4.grid(True)

plt.tight_layout()
plt.show()
```

plot_activation_functions()



3. TensorFlow/Keras

TensorFlow Basics

1. Tensors

- **What it is:** Tensors are the fundamental data structures used in TensorFlow. They are multi-dimensional arrays that hold data for processing in a neural network.

2. Variables

- **What it is:** Variables are the model's parameters (weights and biases) that are updated during training. They are mutable, meaning their values change as the model learns.

3. Operations

- **What it is:** Operations are the mathematical computations performed on tensors, such as addition, multiplication, and matrix operations.

Keras API

1. Sequential API

- **What it is:** A simple way to build neural networks by stacking layers in a linear order (one after another). It's best for straightforward models with a single input and output.

2. Functional API

- **What it is:** A more flexible way to define models in Keras, allowing for non-linear architectures with multiple inputs, outputs, and shared layers.

3. Model Subclassing

- **What it is:** A method for defining custom models by subclassing the Keras `Model` class. This approach allows for complete control over the model architecture and training process.

Model Building

1. Layer Configuration

- **What it is:** The process of defining and arranging the layers in the neural network, including choosing the type of layers (Dense, Convolutional, etc.) and their configurations (e.g., number of units, activation functions).

2. Loss Functions

- **What it is:** A function that measures how well the model's predictions match the actual values. Common loss functions include **binary cross-entropy** for binary classification and **mean squared error** for regression.

3. Optimizers

- **What it is:** Algorithms that adjust the model's weights to minimize the loss function. Examples include **Adam** and **SGD (Stochastic Gradient Descent)**.

3.1 TensorFlow/Keras Implementation - Binary Classification

1. Generate Synthetic Dataset

- **Purpose:** Create a dataset for binary classification where the target variable has two classes.
- **What it does:** Uses `make_classification()` from `sklearn` to generate 1000 samples with 20 features and 2 classes (binary classification). It then splits the dataset into training and testing sets using `train_test_split()`.

2. Create Neural Network Model

- **Purpose:** Build a simple neural network for binary classification.
- **Model Architecture:**
 - **Input Layer:** The model takes 20 features as input.
 - **Hidden Layers:** Two hidden layers:
 - The first hidden layer has 16 neurons and uses the **ReLU** activation function.
 - The second hidden layer has 8 neurons and uses **ReLU** as well.
 - **Dropout Layer:** A dropout rate of 20% is applied after the first hidden layer to prevent overfitting.
 - **Output Layer:** The output layer has 1 neuron with the **sigmoid** activation function, which is suitable for binary classification (outputs a probability between 0 and 1).
- **Compilation:** The model is compiled using the **Adam** optimizer, **binary_crossentropy** loss (common for binary classification), and **accuracy** as the evaluation metric.

3. Train the Model

- **Purpose:** Train the neural network on the dataset.
- **What it does:** The model is trained for 50 epochs using a batch size of 32. 20% of the training data is used for validation during training. The training progress is displayed without verbosity.

4. Plot Training History

- **Purpose:** Visualize the training and validation loss and accuracy over the epochs.
- **Loss Plot:** Shows how the training and validation loss decrease over time, indicating how well the model is learning.
- **Accuracy Plot:** Displays the training and validation accuracy over time, showing how well the model is performing on both the training and validation datasets.

```
In [65]: # Cell 3: TensorFlow/Keras Implementation - Binary Classification
        """
        Implementation of a simple neural network for binary classification
```

```

using TensorFlow/Keras.
"""

# Generate synthetic dataset
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create model
model = keras.Sequential([
    keras.layers.Dense(16, activation='relu', input_shape=(20,)),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(8, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# Train model
history = model.fit(X_train, y_train,
                    epochs=50,
                    batch_size=32,
                    validation_split=0.2,
                    verbose=0)

# Plot training history
def plot_training_history(history):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

    ax1.plot(history.history['loss'], label='Training Loss')
    ax1.plot(history.history['val_loss'], label='Validation Loss')
    ax1.set_title('Model Loss')
    ax1.set_xlabel('Epoch')
    ax1.set_ylabel('Loss')
    ax1.legend()

    ax2.plot(history.history['accuracy'], label='Training Accuracy')
    ax2.plot(history.history['val_accuracy'], label='Validation Accuracy')
    ax2.set_title('Model Accuracy')
    ax2.set_xlabel('Epoch')
    ax2.set_ylabel('Accuracy')
    ax2.legend()

    plt.tight_layout()
    plt.show()

plot_training_history(history)

```



4. PyTorch Introduction

4.1. PyTorch Implementation - Multi-class Classification

PyTorch Basics

1. Tensors

- **What it is:** Like TensorFlow, PyTorch uses tensors as the basic data structure for storing data. Tensors can be used on both CPUs and GPUs for efficient computation.

2. Autograd

- **What it is:** A system in PyTorch that automatically computes gradients for backpropagation. It tracks the operations performed on tensors and calculates the necessary gradients during training.

3. Neural Network Modules

- **What it is:** Predefined building blocks in PyTorch, like layers and loss functions, that help create neural networks. These modules are part of the `torch.nn` package.

Model Development

1. Dataset/DataLoader

- **What it is:** PyTorch provides classes to manage and load datasets. The `Dataset` class represents the data, and `DataLoader` helps load the data in batches for efficient training.

2. Training Loops

- **What it is:** A loop that iterates over the dataset, performs forward propagation, calculates loss, applies backpropagation, and updates the model's parameters.

3. Model Evaluation

- **What it is:** The process of assessing the model's performance on a test dataset. Evaluation metrics such as accuracy or loss are used to understand how well the model is performing.

PyTorch Implementation - Multi-class Classification

1. Load and Preprocess the Dataset

- **Purpose:** Use the digits dataset for multi-class classification (recognizing digits 0-9).
- **What it does:**
 - Loads the dataset using `load_digits()` from `sklearn`, which includes 8x8 pixel images of handwritten digits.
 - Scales the features using `StandardScaler()` to standardize the data to have a mean of 0 and a variance of 1.

2. Convert Data to PyTorch Tensors

- **Purpose:** Convert the dataset into a format that PyTorch can use.
- **What it does:**
 - Converts the scaled features (`X`) into a PyTorch tensor of type `FloatTensor` for model input.
 - Converts the target labels (`y`) into a PyTorch tensor of type `LongTensor` for classification.

3. Create Dataset and DataLoader

- **Purpose:** Create a custom dataset class and a data loader for batching the data.
- **DigitsDataset Class:**
 - Defines a custom dataset for the digits data.
 - Implements methods to retrieve the size of the dataset (`__len__`) and fetch a sample (`__getitem__`).
- **DataLoader:**
 - Wraps the `DigitsDataset` and enables batch processing with a batch size of 32 and random shuffling of the data for training.

4. Define the Neural Network Model

- **Purpose:** Build a neural network for multi-class classification.
- **Model Architecture:**
 - The model has three layers:
 - **Layer 1:** Fully connected layer with 64 input features and 32 output neurons.
 - **Layer 2:** Fully connected layer with 32 input neurons and 16 output neurons.
 - **Layer 3:** Fully connected layer with 16 input neurons and 10 output neurons (since there are 10 classes, representing digits 0-9).
 - **Activation Function:** Uses the **ReLU** activation function after the first two layers.

5. Training Function

- **Purpose:** Train the neural network.
- **What it does:**
 - In each epoch, the model performs the following steps:
 - A. Clears the previous gradients using `optimizer.zero_grad()`.
 - B. Makes predictions using the model with `model(batch_X)`.

- C. Computes the loss using `CrossEntropyLoss()` (common for multi-class classification).
 - D. Backpropagates the loss using `loss.backward()`.
 - E. Updates the model weights using `optimizer.step()`.
 - The function returns a list of losses at each epoch.
6. **Train the Model**
- **Purpose:** Create and train the multi-class classification model.
 - **What it does:**
 - Creates an instance of the `MultiClassNN` model.
 - Uses the **Adam** optimizer and **CrossEntropyLoss** for multi-class classification.
 - Trains the model for 50 epochs using the `train_pytorch_model` function and stores the losses.
7. **Plot Training Loss**
- **Purpose:** Visualize how the loss changes over the training epochs.
 - **What it does:**
 - Plots the training loss over the 50 epochs to monitor how well the model is learning.
 - The loss should generally decrease over time, indicating that the model is improving.

```
In [66]: # PyTorch Implementation - Multi-class Classification
        """
        Implementation of a neural network for multi-class classification
        using PyTorch.
        """

        # Load digits dataset
        digits = load_digits()
        X = digits.data
        y = digits.target

        # Scale the features
        scaler = StandardScaler()
        X = scaler.fit_transform(X)

        # Convert to PyTorch tensors
        X = torch.FloatTensor(X)
        y = torch.LongTensor(y)

        # Create dataset and dataloader
        class DigitsDataset(Dataset):
            def __init__(self, X, y):
                self.X = X
                self.y = y

            def __len__(self):
                return len(self.X)

            def __getitem__(self, idx):
                return self.X[idx], self.y[idx]

        # Create model
        class MultiClassNN(nn.Module):
            def __init__(self):
                super(MultiClassNN, self).__init__()
                self.layer1 = nn.Linear(64, 32)
                self.layer2 = nn.Linear(32, 16)
                self.layer3 = nn.Linear(16, 10)
                self.relu = nn.ReLU()

            def forward(self, x):
                x = self.relu(self.layer1(x))
                x = self.relu(self.layer2(x))
                x = self.layer3(x)
                return x

        # Training function
        def train_pytorch_model(model, train_loader, criterion, optimizer, num_epochs):
            losses = []
            for epoch in range(num_epochs):
                for batch_X, batch_y in train_loader:
                    optimizer.zero_grad()
                    outputs = model(batch_X)
                    loss = criterion(outputs, batch_y)
                    loss.backward()
                    optimizer.step()
                losses.append(loss.item())
            return losses

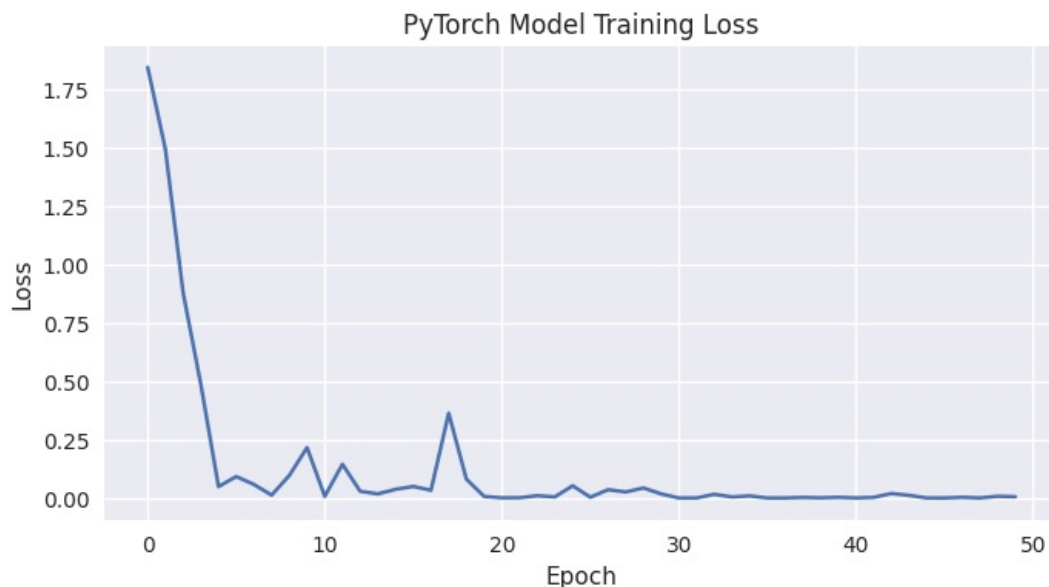
        # Create and train model
```

```
dataset = DigitsDataset(X, y)
train_loader = DataLoader(dataset, batch_size=32, shuffle=True)

model = MultiClassNN()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters())

losses = train_pytorch_model(model, train_loader, criterion, optimizer, 50)

# Plot training loss
plt.figure(figsize=(8, 4))
plt.plot(losses)
plt.title('PyTorch Model Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.show()
```



4.2. Regression Example with Neural Network

1. Generate Synthetic Regression Dataset

- **Purpose:** Create a dataset with a continuous target variable for regression.
- **What it does:** Uses `make_regression()` from `sklearn` to generate 1000 samples with 10 features. It also adds a small amount of noise for realism.
- **Training and Test Split:** Splits the data into training and testing sets using `train_test_split()`.

2. Create Regression Model

- **Purpose:** Build a neural network model for regression.
- **Model Architecture:**
 - **Input Layer:** The model takes 10 features as input.
 - **Hidden Layers:** Two hidden layers with 16 and 8 neurons, both using the **ReLU** activation function.
 - **Output Layer:** The output layer has 1 neuron since it's a regression problem, producing a continuous value.
- **Compilation:** The model is compiled using the **Adam** optimizer, **Mean Squared Error (MSE)** as the loss function (common for regression), and **Mean Absolute Error (MAE)** for additional evaluation.

3. Train the Regression Model

- **Purpose:** Train the model on the training data.
- **What it does:** The model is trained for 50 epochs using a batch size of 32 and 20% of the data reserved for validation. The training progress is shown without verbosity.

4. Plot Regression Results

- **Purpose:** Visualize the model's performance.
- **Loss Plot:** Shows how the training and validation loss (MSE) decrease over time, indicating the model's learning progress.
- **Prediction vs Actual Plot:** Compares predicted values to actual values on the test set. A red dashed line represents the perfect prediction (where predicted values exactly match actual

values).

```
In [67]: # Regression Example
        """
        Implementation of a neural network for regression using TensorFlow/Keras.
        """

        # Generate synthetic regression dataset
        X_reg, y_reg = make_regression(n_samples=1000, n_features=10, noise=0.1, random_state=42)
        X_train, X_test, y_train, y_test = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)

        # Create regression model
        reg_model = keras.Sequential([
            keras.layers.Dense(16, activation='relu', input_shape=(10,)),
            keras.layers.Dense(8, activation='relu'),
            keras.layers.Dense(1)
        ])

        reg_model.compile(optimizer='adam', loss='mse', metrics=['mae'])

        # Train regression model
        reg_history = reg_model.fit(X_train, y_train,
                                    epochs=50,
                                    batch_size=32,
                                    validation_split=0.2,
                                    verbose=0)

        # Plot regression results
        def plot_regression_results():
            # Make predictions
            y_pred = reg_model.predict(X_test)

            plt.figure(figsize=(10, 4))

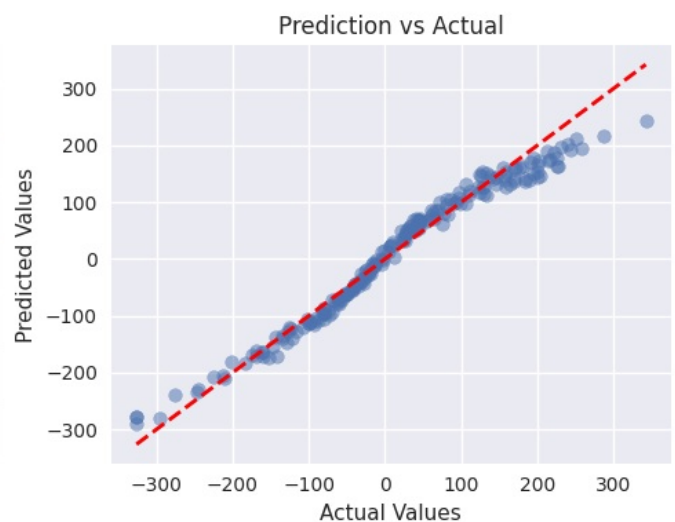
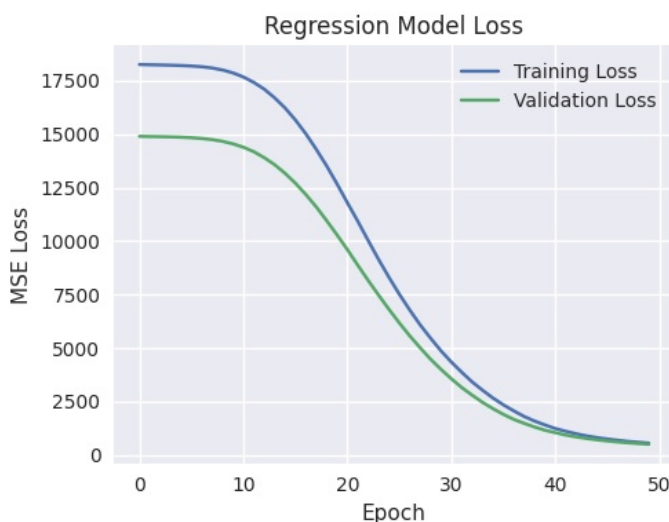
            plt.subplot(1, 2, 1)
            plt.plot(reg_history.history['loss'], label='Training Loss')
            plt.plot(reg_history.history['val_loss'], label='Validation Loss')
            plt.title('Regression Model Loss')
            plt.xlabel('Epoch')
            plt.ylabel('MSE Loss')
            plt.legend()

            plt.subplot(1, 2, 2)
            plt.scatter(y_test, y_pred, alpha=0.5)
            plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
            plt.title('Prediction vs Actual')
            plt.xlabel('Actual Values')
            plt.ylabel('Predicted Values')

            plt.tight_layout()
            plt.show()

        plot_regression_results()
```

7/7 0s 14ms/step



5. Simple Neural Networks

A **simple neural network** is a model inspired by the human brain, designed to recognize patterns and make predictions. It consists of layers of connected neurons, where each neuron performs a

mathematical operation to process input data and produce an output.

Implementation

1. Binary Classification

- **What it is:**

Binary classification is the task of categorizing data into one of two possible outcomes or classes (e.g., spam or not spam, cancerous or not cancerous).

- **In this code:**

A neural network is trained to predict whether breast cancer data belongs to one of two classes: malignant (1) or benign (0). This is done using a **sigmoid activation function** in the final layer of the network, which outputs values between 0 and 1, representing the probability of belonging to one class.

2. Multiclass Classification

- **What it is:**

Multiclass classification involves predicting one of several possible classes (more than two classes).

- **In this code:**

The neural network is trained to classify data into one of four classes using a **softmax activation function** in the output layer. This function assigns probabilities to each class, and the class with the highest probability is selected as the prediction.

3. Regression

- **What it is:**

Regression is a type of problem where the goal is to predict a continuous value rather than a class (e.g., predicting the price of a house, or the temperature on a given day).

- **In this code:**

A neural network is used to predict continuous values, like the target values in a regression dataset. The output layer has a **single neuron** without an activation function, allowing the model to predict a wide range of values.

Training

1. Batch Processing

- **What it is:**

Batch processing refers to the technique of splitting the dataset into smaller subsets (batches) to train the model. Instead of using the entire dataset at once, the model updates its weights after processing each batch of data.

- **In this code:**

The model is trained on batches of data (e.g., 32 samples at a time) rather than the entire dataset. This helps the model learn more efficiently and speeds up the training process.

2. Epochs

- **What it is:**

An epoch refers to one complete pass of the entire dataset through the neural network. After each epoch, the model's weights are updated based on the training data.

- **In this code:**

The model is trained for multiple epochs (e.g., 100 epochs), meaning the dataset is passed through the model 100 times, allowing the model to learn and improve over time.

3. Early Stopping

- **What it is:**

Early stopping is a technique used to prevent overfitting by stopping training if the model's performance on the validation set does not improve after a certain number of epochs.

- **In this code:**
The model stops training early if the validation loss (a measure of the model's error on unseen data) does not improve for a set number of epochs (patience), thus avoiding unnecessary computations and preventing overfitting.
-

Summary of What They Do in the Code:

- **Binary classification** predicts if the data point belongs to one of two classes.
- **Multiclass classification** predicts which class out of several categories the data point belongs to.
- **Regression** predicts a continuous value.
- **Batch processing** helps train the model faster by using smaller subsets of the data.
- **Epochs** represent the number of times the model learns from the entire dataset.
- **Early stopping** stops training when further learning no longer improves the model's performance, saving time and avoiding overfitting.

These techniques work together to help the neural network learn from data, make predictions, and do so efficiently without overfitting.

5.1 Binary Classification with Early Stopping

This code demonstrates a binary classification task using the Breast Cancer Wisconsin dataset and incorporates early stopping to prevent overfitting.

1. Data Preparation:

- The Breast Cancer dataset is loaded using `sklearn.datasets.load_breast_cancer`.
- The dataset is split into training and test sets using `train_test_split`.
- Feature data (`X`) is standardized using `StandardScaler` to ensure that all features have similar scales.

2. Model Definition:

- A neural network model is defined using Keras:
 - The first dense layer has 16 units with ReLU activation.
 - Batch normalization and dropout layers are added for regularization.
 - The second dense layer has 8 units with ReLU activation.
 - The output layer has 1 unit with a sigmoid activation, suitable for binary classification.

3. Model Compilation:

- The model is compiled with the Adam optimizer, binary cross-entropy loss (since it's a binary classification task), and accuracy as the metric.

4. Callbacks:

- **EarlyStopping:** This callback monitors the validation loss and stops training if it doesn't improve for 10 consecutive epochs, restoring the best weights found.
- **ModelCheckpoint:** This callback saves the model whenever the validation loss improves, using the `.keras` format.

5. Training with Different Batch Sizes:

- The model is trained with three different batch sizes: 16, 32, and 64. For each batch size:
 - The model is trained for 100 epochs with early stopping and checkpointing.
 - The training history (loss and accuracy) is stored for later comparison.

6. Plotting Results:

- Two plots are generated to compare the results for different batch sizes:
 - **Loss Plot:** Shows the training and validation loss for each batch size over the epochs.
 - **Accuracy Plot:** Shows the training and validation accuracy for each batch size over the epochs.

7. Model Evaluation:

- After training, the model is evaluated on the test set (`X_test_scaled`, `y_test`) to compute the final test accuracy.

```
In [68]: # Cell 1: Binary Classification with Early Stopping
        """
        Implementation of binary classification with early stopping
        Dataset: Breast Cancer Wisconsin dataset
        """
```

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

# Load and prepare data
data = load_breast_cancer()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Define model
binary_model = keras.Sequential([
    keras.layers.Dense(16, activation='relu', input_shape=(X_train.shape[1],)),
    keras.layers.BatchNormalization(),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(8, activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(1, activation='sigmoid')
])

# Compile model
binary_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Define callbacks
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)

# Updated checkpoint filepath to use .keras extension
checkpoint = ModelCheckpoint(
    'best_binary_model.keras', # Changed from .h5 to .keras
    monitor='val_loss',
    save_best_only=True
)

# Train with different batch sizes
batch_sizes = [16, 32, 64]
history_dict = {}

# Training loop for different batch sizes
for batch_size in batch_sizes:
    print(f"\nTraining with batch size: {batch_size}")
    history = binary_model.fit(
        X_train_scaled, y_train,
        epochs=100,
        batch_size=batch_size,
        validation_split=0.2,
        callbacks=[early_stopping, checkpoint],
        verbose=1 # Changed to 1 to see progress
    )
    history_dict[f'batch_{batch_size}'] = history.history

# Create figure for plotting
plt.figure(figsize=(15, 5))

# Plot training and validation loss
plt.subplot(1, 2, 1)
for batch_size in batch_sizes:
    plt.plot(history_dict[f'batch_{batch_size}']['loss'],
             label=f'Training (batch={batch_size})')
    plt.plot(history_dict[f'batch_{batch_size}']['val_loss'],
             label=f'Validation (batch={batch_size})')
plt.title('Model Loss with Different Batch Sizes')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

```

```

# Plot training and validation accuracy
plt.subplot(1, 2, 2)
for batch_size in batch_sizes:
    plt.plot(history_dict[f'batch_{batch_size}']['accuracy'],
             label=f'Training (batch={batch_size})')
    plt.plot(history_dict[f'batch_{batch_size}']['val_accuracy'],
             label=f'Validation (batch={batch_size})')
plt.title('Model Accuracy with Different Batch Sizes')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.tight_layout()
plt.show()

# Evaluate the model on test set
test_loss, test_accuracy = binary_model.evaluate(X_test_scaled, y_test, verbose=0)
print(f"\nTest accuracy: {test_accuracy:.4f}")

```

Training with batch size: 16

```

Epoch 1/100
23/23 8s 153ms/step - accuracy: 0.4949 - loss: 0.9005 - val_accuracy: 0.7363 - val_loss: 0.5542
Epoch 2/100
23/23 0s 5ms/step - accuracy: 0.6816 - loss: 0.5958 - val_accuracy: 0.7692 - val_loss: 0.4747
Epoch 3/100
23/23 0s 5ms/step - accuracy: 0.7139 - loss: 0.5095 - val_accuracy: 0.8462 - val_loss: 0.4178
Epoch 4/100
23/23 0s 5ms/step - accuracy: 0.7640 - loss: 0.4876 - val_accuracy: 0.8462 - val_loss: 0.3748
Epoch 5/100
23/23 0s 5ms/step - accuracy: 0.8704 - loss: 0.3747 - val_accuracy: 0.8901 - val_loss: 0.3427
Epoch 6/100
23/23 0s 5ms/step - accuracy: 0.8478 - loss: 0.3815 - val_accuracy: 0.9121 - val_loss: 0.3152
Epoch 7/100
23/23 0s 5ms/step - accuracy: 0.8872 - loss: 0.3007 - val_accuracy: 0.9451 - val_loss: 0.2926
Epoch 8/100
23/23 0s 5ms/step - accuracy: 0.8871 - loss: 0.2880 - val_accuracy: 0.9451 - val_loss: 0.2749
Epoch 9/100
23/23 0s 5ms/step - accuracy: 0.8647 - loss: 0.3248 - val_accuracy: 0.9451 - val_loss: 0.2631
Epoch 10/100
23/23 0s 5ms/step - accuracy: 0.8874 - loss: 0.2946 - val_accuracy: 0.9231 - val_loss: 0.2508
Epoch 11/100
23/23 0s 5ms/step - accuracy: 0.9069 - loss: 0.2765 - val_accuracy: 0.9231 - val_loss: 0.2415
Epoch 12/100
23/23 0s 5ms/step - accuracy: 0.8834 - loss: 0.3200 - val_accuracy: 0.9341 - val_loss: 0.2303
Epoch 13/100
23/23 0s 5ms/step - accuracy: 0.9100 - loss: 0.2445 - val_accuracy: 0.9341 - val_loss: 0.2217
Epoch 14/100
23/23 0s 5ms/step - accuracy: 0.9482 - loss: 0.2048 - val_accuracy: 0.9231 - val_loss: 0.2129
Epoch 15/100
23/23 0s 5ms/step - accuracy: 0.8846 - loss: 0.2624 - val_accuracy: 0.9231 - val_loss: 0.2057
Epoch 16/100
23/23 0s 5ms/step - accuracy: 0.9081 - loss: 0.2710 - val_accuracy: 0.9341 - val_loss: 0.1988
Epoch 17/100
23/23 0s 5ms/step - accuracy: 0.9306 - loss: 0.2048 - val_accuracy: 0.9341 - val_loss: 0.1933
Epoch 18/100
23/23 0s 5ms/step - accuracy: 0.9180 - loss: 0.1898 - val_accuracy: 0.9451 - val_loss: 0.1824
Epoch 19/100
23/23 0s 5ms/step - accuracy: 0.9406 - loss: 0.1655 - val_accuracy: 0.9341 - val_loss: 0.1762
Epoch 20/100
23/23 0s 5ms/step - accuracy: 0.9322 - loss: 0.1939 - val_accuracy: 0.9341 - val_loss: 0.1715
Epoch 21/100
23/23 0s 5ms/step - accuracy: 0.9421 - loss: 0.1917 - val_accuracy: 0.9231 - val_loss: 0.1659
Epoch 22/100
23/23 0s 5ms/step - accuracy: 0.9514 - loss: 0.1608 - val_accuracy: 0.9341 - val_loss: 0.1628
Epoch 23/100
23/23 0s 5ms/step - accuracy: 0.9651 - loss: 0.1276 - val_accuracy: 0.9451 - val_loss: 0.1585
Epoch 24/100
23/23 0s 5ms/step - accuracy: 0.9523 - loss: 0.1595 - val_accuracy: 0.9560 - val_loss: 0.1567
Epoch 25/100
23/23 0s 5ms/step - accuracy: 0.9506 - loss: 0.1780 - val_accuracy: 0.9560 - val_loss: 0.1549
Epoch 26/100
23/23 0s 5ms/step - accuracy: 0.9522 - loss: 0.1670 - val_accuracy: 0.9560 - val_loss: 0.1469
Epoch 27/100
23/23 0s 5ms/step - accuracy: 0.9386 - loss: 0.1447 - val_accuracy: 0.9560 - val_loss: 0.1423
Epoch 28/100
23/23 0s 5ms/step - accuracy: 0.9609 - loss: 0.1499 - val_accuracy: 0.9560 - val_loss: 0.1400
Epoch 29/100
23/23 0s 5ms/step - accuracy: 0.9269 - loss: 0.1934 - val_accuracy: 0.9560 - val_loss: 0.1381
Epoch 30/100
23/23 0s 5ms/step - accuracy: 0.9377 - loss: 0.1743 - val_accuracy: 0.9560 - val_loss: 0.1361
Epoch 31/100
23/23 0s 3ms/step - accuracy: 0.9597 - loss: 0.1223 - val_accuracy: 0.9560 - val_loss: 0.1363
Epoch 32/100

```

23/23 0s 5ms/step - accuracy: 0.9599 - loss: 0.1339 - val_accuracy: 0.9560 - val_loss: 0.1327
Epoch 33/100
23/23 0s 5ms/step - accuracy: 0.9434 - loss: 0.1340 - val_accuracy: 0.9560 - val_loss: 0.1316
Epoch 34/100
23/23 0s 5ms/step - accuracy: 0.9331 - loss: 0.1617 - val_accuracy: 0.9560 - val_loss: 0.1267
Epoch 35/100
23/23 0s 5ms/step - accuracy: 0.9378 - loss: 0.1713 - val_accuracy: 0.9451 - val_loss: 0.1270
Epoch 36/100
23/23 0s 5ms/step - accuracy: 0.9513 - loss: 0.1651 - val_accuracy: 0.9560 - val_loss: 0.1235
Epoch 37/100
23/23 0s 5ms/step - accuracy: 0.9383 - loss: 0.1527 - val_accuracy: 0.9560 - val_loss: 0.1188
Epoch 38/100
23/23 0s 6ms/step - accuracy: 0.9634 - loss: 0.1204 - val_accuracy: 0.9670 - val_loss: 0.1151
Epoch 39/100
23/23 0s 5ms/step - accuracy: 0.9569 - loss: 0.1460 - val_accuracy: 0.9670 - val_loss: 0.1123
Epoch 40/100
23/23 0s 5ms/step - accuracy: 0.9764 - loss: 0.0948 - val_accuracy: 0.9670 - val_loss: 0.1102
Epoch 41/100
23/23 0s 5ms/step - accuracy: 0.9632 - loss: 0.1234 - val_accuracy: 0.9560 - val_loss: 0.1087
Epoch 42/100
23/23 0s 3ms/step - accuracy: 0.9551 - loss: 0.1366 - val_accuracy: 0.9341 - val_loss: 0.1100
Epoch 43/100
23/23 0s 6ms/step - accuracy: 0.9526 - loss: 0.1490 - val_accuracy: 0.9560 - val_loss: 0.1076
Epoch 44/100
23/23 0s 6ms/step - accuracy: 0.9573 - loss: 0.1292 - val_accuracy: 0.9560 - val_loss: 0.1069
Epoch 45/100
23/23 0s 5ms/step - accuracy: 0.9535 - loss: 0.1426 - val_accuracy: 0.9780 - val_loss: 0.1043
Epoch 46/100
23/23 0s 3ms/step - accuracy: 0.9407 - loss: 0.1566 - val_accuracy: 0.9451 - val_loss: 0.1101
Epoch 47/100
23/23 0s 3ms/step - accuracy: 0.9832 - loss: 0.0801 - val_accuracy: 0.9670 - val_loss: 0.1077
Epoch 48/100
23/23 0s 3ms/step - accuracy: 0.9344 - loss: 0.2154 - val_accuracy: 0.9670 - val_loss: 0.1066
Epoch 49/100
23/23 0s 3ms/step - accuracy: 0.9724 - loss: 0.0742 - val_accuracy: 0.9670 - val_loss: 0.1052
Epoch 50/100
23/23 0s 3ms/step - accuracy: 0.9648 - loss: 0.1221 - val_accuracy: 0.9670 - val_loss: 0.1049
Epoch 51/100
23/23 0s 5ms/step - accuracy: 0.9832 - loss: 0.0868 - val_accuracy: 0.9670 - val_loss: 0.1042
Epoch 52/100
23/23 0s 5ms/step - accuracy: 0.9643 - loss: 0.0941 - val_accuracy: 0.9670 - val_loss: 0.1033
Epoch 53/100
23/23 0s 5ms/step - accuracy: 0.9841 - loss: 0.1087 - val_accuracy: 0.9670 - val_loss: 0.1021
Epoch 54/100
23/23 0s 3ms/step - accuracy: 0.9798 - loss: 0.1003 - val_accuracy: 0.9670 - val_loss: 0.1059
Epoch 55/100
23/23 0s 3ms/step - accuracy: 0.9757 - loss: 0.0988 - val_accuracy: 0.9670 - val_loss: 0.1070
Epoch 56/100
23/23 0s 5ms/step - accuracy: 0.9535 - loss: 0.1475 - val_accuracy: 0.9670 - val_loss: 0.1017
Epoch 57/100
23/23 0s 3ms/step - accuracy: 0.9015 - loss: 0.2843 - val_accuracy: 0.9670 - val_loss: 0.1038
Epoch 58/100
23/23 0s 3ms/step - accuracy: 0.9754 - loss: 0.0977 - val_accuracy: 0.9670 - val_loss: 0.1026
Epoch 59/100
23/23 0s 5ms/step - accuracy: 0.9728 - loss: 0.0958 - val_accuracy: 0.9670 - val_loss: 0.1013
Epoch 60/100
23/23 0s 5ms/step - accuracy: 0.9685 - loss: 0.0971 - val_accuracy: 0.9780 - val_loss: 0.0995
Epoch 61/100
23/23 0s 5ms/step - accuracy: 0.9670 - loss: 0.0981 - val_accuracy: 0.9780 - val_loss: 0.0988
Epoch 62/100
23/23 0s 3ms/step - accuracy: 0.9656 - loss: 0.1194 - val_accuracy: 0.9670 - val_loss: 0.1014
Epoch 63/100
23/23 0s 3ms/step - accuracy: 0.9633 - loss: 0.0929 - val_accuracy: 0.9670 - val_loss: 0.1021
Epoch 64/100
23/23 0s 3ms/step - accuracy: 0.9680 - loss: 0.0786 - val_accuracy: 0.9670 - val_loss: 0.1022
Epoch 65/100
23/23 0s 3ms/step - accuracy: 0.9626 - loss: 0.1311 - val_accuracy: 0.9560 - val_loss: 0.1030
Epoch 66/100
23/23 0s 3ms/step - accuracy: 0.9752 - loss: 0.0891 - val_accuracy: 0.9780 - val_loss: 0.1000
Epoch 67/100
23/23 0s 5ms/step - accuracy: 0.9634 - loss: 0.0941 - val_accuracy: 0.9780 - val_loss: 0.0977
Epoch 68/100
23/23 0s 5ms/step - accuracy: 0.9694 - loss: 0.0785 - val_accuracy: 0.9780 - val_loss: 0.0943
Epoch 69/100
23/23 0s 5ms/step - accuracy: 0.9529 - loss: 0.1470 - val_accuracy: 0.9890 - val_loss: 0.0911
Epoch 70/100
23/23 0s 5ms/step - accuracy: 0.9847 - loss: 0.0563 - val_accuracy: 0.9780 - val_loss: 0.0881
Epoch 71/100
23/23 0s 3ms/step - accuracy: 0.9719 - loss: 0.1024 - val_accuracy: 0.9890 - val_loss: 0.0884
Epoch 72/100
23/23 0s 3ms/step - accuracy: 0.9762 - loss: 0.0774 - val_accuracy: 0.9780 - val_loss: 0.0883
Epoch 73/100
23/23 0s 3ms/step - accuracy: 0.9585 - loss: 0.1138 - val_accuracy: 0.9780 - val_loss: 0.0949

Epoch 74/100
23/23 0s 3ms/step - accuracy: 0.9704 - loss: 0.0720 - val_accuracy: 0.9780 - val_loss: 0.0963
Epoch 75/100
23/23 0s 3ms/step - accuracy: 0.9609 - loss: 0.0918 - val_accuracy: 0.9780 - val_loss: 0.0956
Epoch 76/100
23/23 0s 3ms/step - accuracy: 0.9608 - loss: 0.1168 - val_accuracy: 0.9780 - val_loss: 0.0937
Epoch 77/100
23/23 0s 3ms/step - accuracy: 0.9756 - loss: 0.0646 - val_accuracy: 0.9780 - val_loss: 0.0912
Epoch 78/100
23/23 0s 5ms/step - accuracy: 0.9549 - loss: 0.1334 - val_accuracy: 0.9890 - val_loss: 0.0865
Epoch 79/100
23/23 0s 3ms/step - accuracy: 0.9842 - loss: 0.0625 - val_accuracy: 0.9890 - val_loss: 0.0898
Epoch 80/100
23/23 0s 3ms/step - accuracy: 0.9649 - loss: 0.0880 - val_accuracy: 0.9890 - val_loss: 0.0888
Epoch 81/100
23/23 0s 3ms/step - accuracy: 0.9755 - loss: 0.1071 - val_accuracy: 0.9780 - val_loss: 0.0909
Epoch 82/100
23/23 0s 3ms/step - accuracy: 0.9680 - loss: 0.0897 - val_accuracy: 0.9780 - val_loss: 0.0948
Epoch 83/100
23/23 0s 3ms/step - accuracy: 0.9693 - loss: 0.1255 - val_accuracy: 0.9780 - val_loss: 0.0996
Epoch 84/100
23/23 0s 3ms/step - accuracy: 0.9693 - loss: 0.0868 - val_accuracy: 0.9670 - val_loss: 0.1042
Epoch 85/100
23/23 0s 3ms/step - accuracy: 0.9824 - loss: 0.0883 - val_accuracy: 0.9780 - val_loss: 0.1051
Epoch 86/100
23/23 0s 3ms/step - accuracy: 0.9839 - loss: 0.0620 - val_accuracy: 0.9670 - val_loss: 0.1027
Epoch 87/100
23/23 0s 3ms/step - accuracy: 0.9838 - loss: 0.0816 - val_accuracy: 0.9670 - val_loss: 0.1045
Epoch 88/100
23/23 0s 3ms/step - accuracy: 0.9921 - loss: 0.0532 - val_accuracy: 0.9670 - val_loss: 0.1088

Training with batch size: 32

Epoch 1/100
12/12 3s 30ms/step - accuracy: 0.9836 - loss: 0.0547 - val_accuracy: 0.9890 - val_loss: 0.0875
Epoch 2/100
12/12 0s 4ms/step - accuracy: 0.9684 - loss: 0.0898 - val_accuracy: 0.9890 - val_loss: 0.0894
Epoch 3/100
12/12 0s 4ms/step - accuracy: 0.9694 - loss: 0.0820 - val_accuracy: 0.9780 - val_loss: 0.0910
Epoch 4/100
12/12 0s 4ms/step - accuracy: 0.9814 - loss: 0.0604 - val_accuracy: 0.9780 - val_loss: 0.0920
Epoch 5/100
12/12 0s 4ms/step - accuracy: 0.9832 - loss: 0.0690 - val_accuracy: 0.9780 - val_loss: 0.0930
Epoch 6/100
12/12 0s 4ms/step - accuracy: 0.9912 - loss: 0.0425 - val_accuracy: 0.9780 - val_loss: 0.0962
Epoch 7/100
12/12 0s 4ms/step - accuracy: 0.9838 - loss: 0.0617 - val_accuracy: 0.9780 - val_loss: 0.0959
Epoch 8/100
12/12 0s 4ms/step - accuracy: 0.9741 - loss: 0.0727 - val_accuracy: 0.9780 - val_loss: 0.0968
Epoch 9/100
12/12 0s 4ms/step - accuracy: 0.9574 - loss: 0.1018 - val_accuracy: 0.9780 - val_loss: 0.0960
Epoch 10/100
12/12 0s 4ms/step - accuracy: 0.9800 - loss: 0.0543 - val_accuracy: 0.9780 - val_loss: 0.0928

Training with batch size: 64

Epoch 1/100
6/6 6s 629ms/step - accuracy: 0.9795 - loss: 0.0636 - val_accuracy: 0.9890 - val_loss: 0.0867
Epoch 2/100
6/6 0s 14ms/step - accuracy: 0.9829 - loss: 0.0613 - val_accuracy: 0.9890 - val_loss: 0.0864
Epoch 3/100
6/6 0s 7ms/step - accuracy: 0.9921 - loss: 0.0470 - val_accuracy: 0.9890 - val_loss: 0.0864
Epoch 4/100
6/6 0s 7ms/step - accuracy: 0.9864 - loss: 0.0503 - val_accuracy: 0.9890 - val_loss: 0.0872
Epoch 5/100
6/6 0s 7ms/step - accuracy: 0.9797 - loss: 0.0676 - val_accuracy: 0.9890 - val_loss: 0.0877
Epoch 6/100
6/6 0s 7ms/step - accuracy: 0.9974 - loss: 0.0394 - val_accuracy: 0.9890 - val_loss: 0.0881
Epoch 7/100
6/6 0s 7ms/step - accuracy: 0.9797 - loss: 0.0688 - val_accuracy: 0.9780 - val_loss: 0.0884
Epoch 8/100
6/6 0s 7ms/step - accuracy: 0.9842 - loss: 0.0554 - val_accuracy: 0.9780 - val_loss: 0.0886
Epoch 9/100
6/6 0s 7ms/step - accuracy: 0.9920 - loss: 0.0600 - val_accuracy: 0.9780 - val_loss: 0.0890
Epoch 10/100
6/6 0s 7ms/step - accuracy: 0.9807 - loss: 0.0708 - val_accuracy: 0.9780 - val_loss: 0.0896
Epoch 11/100
6/6 0s 7ms/step - accuracy: 0.9720 - loss: 0.0619 - val_accuracy: 0.9780 - val_loss: 0.0903
Epoch 12/100
6/6 0s 7ms/step - accuracy: 0.9766 - loss: 0.0673 - val_accuracy: 0.9780 - val_loss: 0.0910



Test accuracy: 0.9737

5.2 Multiclass Classification with Learning Rate Schedule

This code demonstrates the implementation of a multiclass classification problem using a custom learning rate schedule.

1. Data Generation:

- A synthetic multiclass dataset is generated using `sklearn.datasets.make_classification`, with 1000 samples, 20 features, and 4 classes. Some features are informative and others are redundant.
- The dataset is split into training and test sets using `train_test_split`.

2. Learning Rate Scheduling:

- A custom learning rate schedule function (`lr_schedule`) is defined to reduce the learning rate during training:
 - The learning rate starts at `0.001`.
 - Every 10 epochs, the learning rate is halved (using an exponential decay function).
- The learning rate schedule is passed to a `LearningRateScheduler` callback during model training.

3. Model Definition:

- A neural network model is defined with the following layers:
 - A dense layer with 32 units and ReLU activation.
 - Batch normalization and dropout for regularization.
 - A second dense layer with 16 units, followed by batch normalization and dropout.
 - The output layer with 4 units and a softmax activation (for multiclass classification).

4. Model Compilation:

- The model is compiled with the Adam optimizer, a learning rate of `0.001`, and `sparse_categorical_crossentropy` loss (for multiclass classification).
- Accuracy is used as a performance metric.

5. Model Training:

- The model is trained using the `fit` method with the following parameters:
 - Training data `X_train`, `y_train`.
 - A batch size of 32 and validation split of 20%.
 - The `LearningRateScheduler` and `EarlyStopping` callbacks are used to adjust the learning rate and stop training early if validation loss doesn't improve.

6. Plotting Results:

- Three plots are generated to visualize the training process:
 - **Loss curve:** Shows both training and validation loss over epochs.
 - **Accuracy curve:** Shows both training and validation accuracy over epochs.
 - **Learning rate schedule:** Plots the learning rate changes over time.

7. Model Evaluation:

- The model is evaluated on the test set (`X_test`, `y_test`) to obtain the final accuracy.

- The test accuracy is printed.

The learning rate schedule helps to improve model training by reducing the learning rate as training progresses, preventing overshooting and promoting finer convergence.

```
In [69]: # Cell 2: Multiclass Classification with Learning Rate Schedule
        """
        Implementation of multiclass classification with learning rate scheduling
        Dataset: Generated multiclass dataset
        """

        import numpy as np
        import matplotlib.pyplot as plt
        from sklearn.datasets import make_classification
        from sklearn.model_selection import train_test_split
        import tensorflow as tf
        from tensorflow import keras
        from tensorflow.keras.callbacks import EarlyStopping
        import math

        # Generate multiclass data
        X_multi, y_multi = make_classification(
            n_samples=1000,
            n_features=20,
            n_informative=15,
            n_redundant=5,
            n_classes=4,
            random_state=42
        )

        X_train, X_test, y_train, y_test = train_test_split(X_multi, y_multi, test_size=0.2, random_state=42)

        # Learning rate schedule
        def lr_schedule(epoch):
            initial_lr = 0.001
            drop = 0.5
            epochs_drop = 10.0
            lr = initial_lr * math.pow(drop, math.floor((1 + epoch) / epochs_drop))
            return lr

        # Define multiclass model
        multiclass_model = keras.Sequential([
            keras.layers.Dense(32, activation='relu', input_shape=(20,)),
            keras.layers.BatchNormalization(),
            keras.layers.Dropout(0.3),
            keras.layers.Dense(16, activation='relu'),
            keras.layers.BatchNormalization(),
            keras.layers.Dropout(0.2),
            keras.layers.Dense(4, activation='softmax')
        ])

        # Compile model
        multiclass_model.compile(
            optimizer=keras.optimizers.Adam(learning_rate=0.001),
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy']
        )

        # Train with learning rate scheduling
        lr_scheduler = keras.callbacks.LearningRateScheduler(lr_schedule)
        early_stopping = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

        history_multi = multiclass_model.fit(
            X_train, y_train,
            epochs=100,
            batch_size=32,
            validation_split=0.2,
            callbacks=[lr_scheduler, early_stopping],
            verbose=1
        )

        # Plot multiclass results
        plt.figure(figsize=(15, 5))

        plt.subplot(1, 3, 1)
        plt.plot(history_multi.history['loss'], label='Training Loss')
        plt.plot(history_multi.history['val_loss'], label='Validation Loss')
        plt.title('Multiclass Model Loss')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.legend()
```

```

plt.subplot(1, 3, 2)
plt.plot(history_multi.history['accuracy'], label='Training Accuracy')
plt.plot(history_multi.history['val_accuracy'], label='Validation Accuracy')
plt.title('Multiclass Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 3, 3)
lr_values = [lr_schedule(epoch) for epoch in range(len(history_multi.history['loss']))]
plt.plot(lr_values)
plt.title('Learning Rate Schedule')
plt.xlabel('Epoch')
plt.ylabel('Learning Rate')

plt.tight_layout()
plt.show()

# Evaluate the model on test set
test_loss, test_accuracy = multiclass_model.evaluate(X_test, y_test, verbose=0)
print(f"\nTest accuracy: {test_accuracy:.4f}")

```

```

Epoch 1/100
20/20 6s 23ms/step - accuracy: 0.2467 - loss: 1.8865 - val_accuracy: 0.2812 - val_loss: 1.6534 - learning_rate: 0.0010
Epoch 2/100
20/20 0s 3ms/step - accuracy: 0.2152 - loss: 1.8049 - val_accuracy: 0.3187 - val_loss: 1.4470 - learning_rate: 0.0010
Epoch 3/100
20/20 0s 3ms/step - accuracy: 0.3745 - loss: 1.5311 - val_accuracy: 0.3625 - val_loss: 1.3599 - learning_rate: 0.0010
Epoch 4/100
20/20 0s 3ms/step - accuracy: 0.3341 - loss: 1.4982 - val_accuracy: 0.3750 - val_loss: 1.3062 - learning_rate: 0.0010
Epoch 5/100
20/20 0s 3ms/step - accuracy: 0.3702 - loss: 1.3659 - val_accuracy: 0.4187 - val_loss: 1.2565 - learning_rate: 0.0010
Epoch 6/100
20/20 0s 3ms/step - accuracy: 0.3787 - loss: 1.3890 - val_accuracy: 0.4812 - val_loss: 1.2145 - learning_rate: 0.0010
Epoch 7/100
20/20 0s 3ms/step - accuracy: 0.3990 - loss: 1.3171 - val_accuracy: 0.5188 - val_loss: 1.1709 - learning_rate: 0.0010
Epoch 8/100
20/20 0s 3ms/step - accuracy: 0.4321 - loss: 1.2722 - val_accuracy: 0.5625 - val_loss: 1.1437 - learning_rate: 0.0010
Epoch 9/100
20/20 0s 3ms/step - accuracy: 0.4026 - loss: 1.2497 - val_accuracy: 0.5750 - val_loss: 1.1170 - learning_rate: 0.0010
Epoch 10/100
20/20 0s 3ms/step - accuracy: 0.4267 - loss: 1.2144 - val_accuracy: 0.5750 - val_loss: 1.1083 - learning_rate: 5.0000e-04
Epoch 11/100
20/20 0s 4ms/step - accuracy: 0.5295 - loss: 1.1565 - val_accuracy: 0.5813 - val_loss: 1.0972 - learning_rate: 5.0000e-04
Epoch 12/100
20/20 0s 4ms/step - accuracy: 0.4658 - loss: 1.2138 - val_accuracy: 0.5813 - val_loss: 1.0876 - learning_rate: 5.0000e-04
Epoch 13/100
20/20 0s 3ms/step - accuracy: 0.4933 - loss: 1.1401 - val_accuracy: 0.5813 - val_loss: 1.0769 - learning_rate: 5.0000e-04
Epoch 14/100
20/20 0s 3ms/step - accuracy: 0.4884 - loss: 1.1654 - val_accuracy: 0.6062 - val_loss: 1.0660 - learning_rate: 5.0000e-04
Epoch 15/100
20/20 0s 3ms/step - accuracy: 0.4898 - loss: 1.1333 - val_accuracy: 0.6000 - val_loss: 1.0550 - learning_rate: 5.0000e-04
Epoch 16/100
20/20 0s 3ms/step - accuracy: 0.4595 - loss: 1.1961 - val_accuracy: 0.5875 - val_loss: 1.0449 - learning_rate: 5.0000e-04
Epoch 17/100
20/20 0s 3ms/step - accuracy: 0.4944 - loss: 1.1251 - val_accuracy: 0.5813 - val_loss: 1.0327 - learning_rate: 5.0000e-04
Epoch 18/100
20/20 0s 3ms/step - accuracy: 0.5522 - loss: 1.0672 - val_accuracy: 0.5875 - val_loss: 1.0257 - learning_rate: 5.0000e-04
Epoch 19/100
20/20 0s 3ms/step - accuracy: 0.5036 - loss: 1.1286 - val_accuracy: 0.5813 - val_loss: 1.0185 - learning_rate: 5.0000e-04
Epoch 20/100
20/20 0s 3ms/step - accuracy: 0.5179 - loss: 1.0754 - val_accuracy: 0.5813 - val_loss: 1.0140 - learning_rate: 2.5000e-04
Epoch 21/100
20/20 0s 3ms/step - accuracy: 0.5172 - loss: 1.0955 - val_accuracy: 0.5938 - val_loss: 1.0083 - learning_rate:

```

2.5000e-04
Epoch 22/100
20/20 0s 3ms/step - accuracy: 0.4906 - loss: 1.1167 - val_accuracy: 0.5938 - val_loss: 1.0042 - learning_rate:
2.5000e-04
Epoch 23/100
20/20 0s 3ms/step - accuracy: 0.5221 - loss: 1.0821 - val_accuracy: 0.5938 - val_loss: 1.0019 - learning_rate:
2.5000e-04
Epoch 24/100
20/20 0s 3ms/step - accuracy: 0.5191 - loss: 1.0807 - val_accuracy: 0.5938 - val_loss: 0.9981 - learning_rate:
2.5000e-04
Epoch 25/100
20/20 0s 3ms/step - accuracy: 0.5478 - loss: 1.0452 - val_accuracy: 0.5938 - val_loss: 0.9945 - learning_rate:
2.5000e-04
Epoch 26/100
20/20 0s 3ms/step - accuracy: 0.5109 - loss: 1.0601 - val_accuracy: 0.6125 - val_loss: 0.9911 - learning_rate:
2.5000e-04
Epoch 27/100
20/20 0s 3ms/step - accuracy: 0.5582 - loss: 1.0396 - val_accuracy: 0.6000 - val_loss: 0.9882 - learning_rate:
2.5000e-04
Epoch 28/100
20/20 0s 3ms/step - accuracy: 0.5386 - loss: 1.0444 - val_accuracy: 0.5938 - val_loss: 0.9851 - learning_rate:
2.5000e-04
Epoch 29/100
20/20 0s 3ms/step - accuracy: 0.5198 - loss: 1.0562 - val_accuracy: 0.6125 - val_loss: 0.9800 - learning_rate:
2.5000e-04
Epoch 30/100
20/20 0s 3ms/step - accuracy: 0.4995 - loss: 1.0660 - val_accuracy: 0.6062 - val_loss: 0.9778 - learning_rate:
1.2500e-04
Epoch 31/100
20/20 0s 4ms/step - accuracy: 0.5742 - loss: 0.9952 - val_accuracy: 0.6125 - val_loss: 0.9748 - learning_rate:
1.2500e-04
Epoch 32/100
20/20 0s 4ms/step - accuracy: 0.5064 - loss: 1.0737 - val_accuracy: 0.6125 - val_loss: 0.9727 - learning_rate:
1.2500e-04
Epoch 33/100
20/20 0s 3ms/step - accuracy: 0.5155 - loss: 1.0487 - val_accuracy: 0.6125 - val_loss: 0.9713 - learning_rate:
1.2500e-04
Epoch 34/100
20/20 0s 3ms/step - accuracy: 0.5221 - loss: 1.0648 - val_accuracy: 0.6125 - val_loss: 0.9683 - learning_rate:
1.2500e-04
Epoch 35/100
20/20 0s 3ms/step - accuracy: 0.5488 - loss: 1.0405 - val_accuracy: 0.6125 - val_loss: 0.9663 - learning_rate:
1.2500e-04
Epoch 36/100
20/20 0s 4ms/step - accuracy: 0.5757 - loss: 0.9861 - val_accuracy: 0.6187 - val_loss: 0.9636 - learning_rate:
1.2500e-04
Epoch 37/100
20/20 0s 3ms/step - accuracy: 0.5107 - loss: 1.0882 - val_accuracy: 0.6187 - val_loss: 0.9620 - learning_rate:
1.2500e-04
Epoch 38/100
20/20 0s 3ms/step - accuracy: 0.5500 - loss: 1.0362 - val_accuracy: 0.6187 - val_loss: 0.9605 - learning_rate:
1.2500e-04
Epoch 39/100
20/20 0s 3ms/step - accuracy: 0.5910 - loss: 1.0068 - val_accuracy: 0.6187 - val_loss: 0.9593 - learning_rate:
1.2500e-04
Epoch 40/100
20/20 0s 3ms/step - accuracy: 0.5328 - loss: 1.0250 - val_accuracy: 0.6187 - val_loss: 0.9593 - learning_rate:
6.2500e-05
Epoch 41/100
20/20 0s 3ms/step - accuracy: 0.5646 - loss: 1.0135 - val_accuracy: 0.6187 - val_loss: 0.9587 - learning_rate:
6.2500e-05
Epoch 42/100
20/20 0s 3ms/step - accuracy: 0.5060 - loss: 1.0526 - val_accuracy: 0.6187 - val_loss: 0.9582 - learning_rate:
6.2500e-05
Epoch 43/100
20/20 0s 3ms/step - accuracy: 0.5568 - loss: 1.0026 - val_accuracy: 0.6187 - val_loss: 0.9578 - learning_rate:
6.2500e-05
Epoch 44/100
20/20 0s 3ms/step - accuracy: 0.5488 - loss: 1.0290 - val_accuracy: 0.6187 - val_loss: 0.9573 - learning_rate:
6.2500e-05
Epoch 45/100
20/20 0s 3ms/step - accuracy: 0.5359 - loss: 1.0385 - val_accuracy: 0.6250 - val_loss: 0.9556 - learning_rate:
6.2500e-05
Epoch 46/100
20/20 0s 3ms/step - accuracy: 0.5870 - loss: 0.9845 - val_accuracy: 0.6250 - val_loss: 0.9551 - learning_rate:
6.2500e-05
Epoch 47/100
20/20 0s 3ms/step - accuracy: 0.5460 - loss: 1.0317 - val_accuracy: 0.6250 - val_loss: 0.9539 - learning_rate:
6.2500e-05
Epoch 48/100
20/20 0s 4ms/step - accuracy: 0.5557 - loss: 1.0502 - val_accuracy: 0.6187 - val_loss: 0.9525 - learning_rate:
6.2500e-05
Epoch 49/100

20/20 0s 3ms/step - accuracy: 0.5545 - loss: 1.0343 - val_accuracy: 0.6187 - val_loss: 0.9514 - learning_rate: 6.2500e-05
Epoch 50/100
20/20 0s 3ms/step - accuracy: 0.5299 - loss: 1.0346 - val_accuracy: 0.6187 - val_loss: 0.9507 - learning_rate: 3.1250e-05
Epoch 51/100
20/20 0s 3ms/step - accuracy: 0.5336 - loss: 1.0426 - val_accuracy: 0.6250 - val_loss: 0.9496 - learning_rate: 3.1250e-05
Epoch 52/100
20/20 0s 3ms/step - accuracy: 0.5979 - loss: 0.9531 - val_accuracy: 0.6250 - val_loss: 0.9484 - learning_rate: 3.1250e-05
Epoch 53/100
20/20 0s 3ms/step - accuracy: 0.5386 - loss: 1.0140 - val_accuracy: 0.6250 - val_loss: 0.9477 - learning_rate: 3.1250e-05
Epoch 54/100
20/20 0s 3ms/step - accuracy: 0.5771 - loss: 1.0036 - val_accuracy: 0.6250 - val_loss: 0.9470 - learning_rate: 3.1250e-05
Epoch 55/100
20/20 0s 3ms/step - accuracy: 0.5579 - loss: 1.0283 - val_accuracy: 0.6250 - val_loss: 0.9459 - learning_rate: 3.1250e-05
Epoch 56/100
20/20 0s 3ms/step - accuracy: 0.5743 - loss: 0.9929 - val_accuracy: 0.6250 - val_loss: 0.9455 - learning_rate: 3.1250e-05
Epoch 57/100
20/20 0s 3ms/step - accuracy: 0.5625 - loss: 1.0228 - val_accuracy: 0.6250 - val_loss: 0.9453 - learning_rate: 3.1250e-05
Epoch 58/100
20/20 0s 4ms/step - accuracy: 0.5859 - loss: 1.0239 - val_accuracy: 0.6187 - val_loss: 0.9445 - learning_rate: 3.1250e-05
Epoch 59/100
20/20 0s 3ms/step - accuracy: 0.5410 - loss: 1.0219 - val_accuracy: 0.6250 - val_loss: 0.9443 - learning_rate: 3.1250e-05
Epoch 60/100
20/20 0s 3ms/step - accuracy: 0.5548 - loss: 0.9929 - val_accuracy: 0.6187 - val_loss: 0.9436 - learning_rate: 1.5625e-05
Epoch 61/100
20/20 0s 3ms/step - accuracy: 0.5358 - loss: 1.0449 - val_accuracy: 0.6187 - val_loss: 0.9438 - learning_rate: 1.5625e-05
Epoch 62/100
20/20 0s 3ms/step - accuracy: 0.5612 - loss: 0.9942 - val_accuracy: 0.6250 - val_loss: 0.9444 - learning_rate: 1.5625e-05
Epoch 63/100
20/20 0s 3ms/step - accuracy: 0.5040 - loss: 1.0443 - val_accuracy: 0.6250 - val_loss: 0.9440 - learning_rate: 1.5625e-05
Epoch 64/100
20/20 0s 3ms/step - accuracy: 0.5110 - loss: 1.0548 - val_accuracy: 0.6250 - val_loss: 0.9437 - learning_rate: 1.5625e-05
Epoch 65/100
20/20 0s 3ms/step - accuracy: 0.5978 - loss: 0.9923 - val_accuracy: 0.6250 - val_loss: 0.9431 - learning_rate: 1.5625e-05
Epoch 66/100
20/20 0s 3ms/step - accuracy: 0.5950 - loss: 1.0464 - val_accuracy: 0.6250 - val_loss: 0.9426 - learning_rate: 1.5625e-05
Epoch 67/100
20/20 0s 3ms/step - accuracy: 0.5544 - loss: 1.0295 - val_accuracy: 0.6250 - val_loss: 0.9424 - learning_rate: 1.5625e-05
Epoch 68/100
20/20 0s 3ms/step - accuracy: 0.5197 - loss: 1.0565 - val_accuracy: 0.6313 - val_loss: 0.9418 - learning_rate: 1.5625e-05
Epoch 69/100
20/20 0s 3ms/step - accuracy: 0.5546 - loss: 0.9809 - val_accuracy: 0.6250 - val_loss: 0.9418 - learning_rate: 1.5625e-05
Epoch 70/100
20/20 0s 3ms/step - accuracy: 0.5776 - loss: 0.9641 - val_accuracy: 0.6250 - val_loss: 0.9411 - learning_rate: 7.8125e-06
Epoch 71/100
20/20 0s 3ms/step - accuracy: 0.5782 - loss: 0.9580 - val_accuracy: 0.6250 - val_loss: 0.9415 - learning_rate: 7.8125e-06
Epoch 72/100
20/20 0s 3ms/step - accuracy: 0.5159 - loss: 1.0408 - val_accuracy: 0.6187 - val_loss: 0.9418 - learning_rate: 7.8125e-06
Epoch 73/100
20/20 0s 3ms/step - accuracy: 0.6097 - loss: 0.9829 - val_accuracy: 0.6250 - val_loss: 0.9421 - learning_rate: 7.8125e-06
Epoch 74/100
20/20 0s 3ms/step - accuracy: 0.5641 - loss: 1.0052 - val_accuracy: 0.6250 - val_loss: 0.9418 - learning_rate: 7.8125e-06
Epoch 75/100
20/20 0s 3ms/step - accuracy: 0.5690 - loss: 0.9799 - val_accuracy: 0.6250 - val_loss: 0.9421 - learning_rate: 7.8125e-06
Epoch 76/100
20/20 0s 3ms/step - accuracy: 0.5792 - loss: 0.9986 - val_accuracy: 0.6313 - val_loss: 0.9413 - learning_rate: 7.8125e-06

Epoch 77/100
20/20 0s 3ms/step - accuracy: 0.5479 - loss: 0.9934 - val_accuracy: 0.6250 - val_loss: 0.9403 - learning_rate: 7.8125e-06

Epoch 78/100
20/20 0s 3ms/step - accuracy: 0.5567 - loss: 1.0106 - val_accuracy: 0.6250 - val_loss: 0.9405 - learning_rate: 7.8125e-06

Epoch 79/100
20/20 0s 3ms/step - accuracy: 0.5809 - loss: 1.0179 - val_accuracy: 0.6250 - val_loss: 0.9406 - learning_rate: 7.8125e-06

Epoch 80/100
20/20 0s 3ms/step - accuracy: 0.5690 - loss: 0.9611 - val_accuracy: 0.6187 - val_loss: 0.9414 - learning_rate: 3.9063e-06

Epoch 81/100
20/20 0s 3ms/step - accuracy: 0.5800 - loss: 0.9933 - val_accuracy: 0.6187 - val_loss: 0.9414 - learning_rate: 3.9063e-06

Epoch 82/100
20/20 0s 3ms/step - accuracy: 0.5759 - loss: 0.9745 - val_accuracy: 0.6187 - val_loss: 0.9413 - learning_rate: 3.9063e-06

Epoch 83/100
20/20 0s 3ms/step - accuracy: 0.5689 - loss: 1.0463 - val_accuracy: 0.6187 - val_loss: 0.9405 - learning_rate: 3.9063e-06

Epoch 84/100
20/20 0s 3ms/step - accuracy: 0.6068 - loss: 0.9913 - val_accuracy: 0.6187 - val_loss: 0.9409 - learning_rate: 3.9063e-06

Epoch 85/100
20/20 0s 3ms/step - accuracy: 0.5727 - loss: 0.9831 - val_accuracy: 0.6187 - val_loss: 0.9412 - learning_rate: 3.9063e-06

Epoch 86/100
20/20 0s 3ms/step - accuracy: 0.5672 - loss: 0.9816 - val_accuracy: 0.6187 - val_loss: 0.9408 - learning_rate: 3.9063e-06

Epoch 87/100
20/20 0s 3ms/step - accuracy: 0.5762 - loss: 1.0195 - val_accuracy: 0.6187 - val_loss: 0.9399 - learning_rate: 3.9063e-06

Epoch 88/100
20/20 0s 3ms/step - accuracy: 0.5450 - loss: 1.0417 - val_accuracy: 0.6187 - val_loss: 0.9406 - learning_rate: 3.9063e-06

Epoch 89/100
20/20 0s 3ms/step - accuracy: 0.5656 - loss: 0.9971 - val_accuracy: 0.6313 - val_loss: 0.9401 - learning_rate: 3.9063e-06

Epoch 90/100
20/20 0s 5ms/step - accuracy: 0.5734 - loss: 0.9861 - val_accuracy: 0.6313 - val_loss: 0.9397 - learning_rate: 1.9531e-06

Epoch 91/100
20/20 0s 4ms/step - accuracy: 0.5709 - loss: 0.9836 - val_accuracy: 0.6313 - val_loss: 0.9397 - learning_rate: 1.9531e-06

Epoch 92/100
20/20 0s 4ms/step - accuracy: 0.5836 - loss: 0.9875 - val_accuracy: 0.6187 - val_loss: 0.9396 - learning_rate: 1.9531e-06

Epoch 93/100
20/20 0s 4ms/step - accuracy: 0.6157 - loss: 0.9687 - val_accuracy: 0.6250 - val_loss: 0.9393 - learning_rate: 1.9531e-06

Epoch 94/100
20/20 0s 4ms/step - accuracy: 0.5742 - loss: 0.9742 - val_accuracy: 0.6250 - val_loss: 0.9383 - learning_rate: 1.9531e-06

Epoch 95/100
20/20 0s 3ms/step - accuracy: 0.5701 - loss: 0.9750 - val_accuracy: 0.6250 - val_loss: 0.9389 - learning_rate: 1.9531e-06

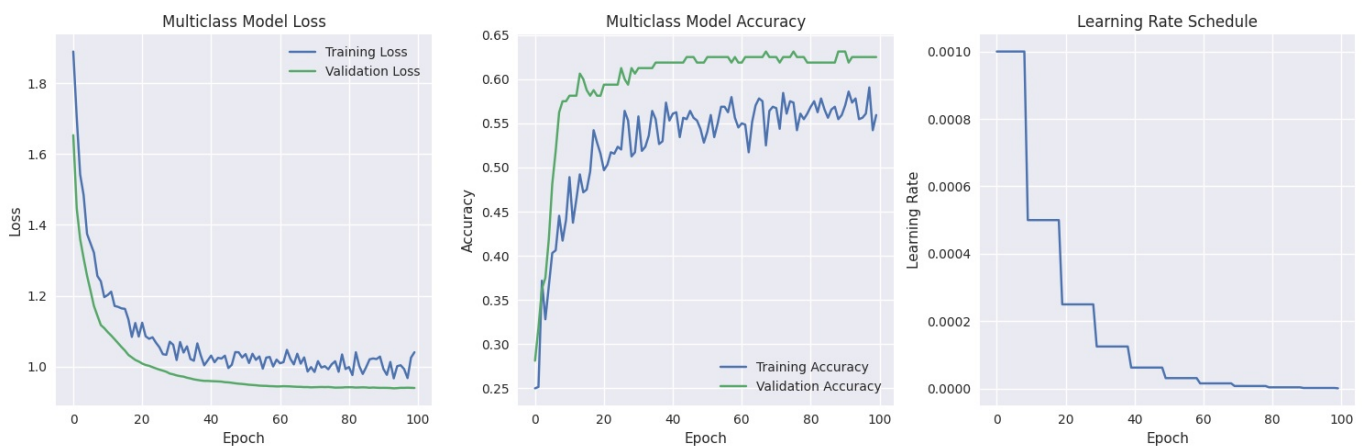
Epoch 96/100
20/20 0s 4ms/step - accuracy: 0.5613 - loss: 0.9798 - val_accuracy: 0.6250 - val_loss: 0.9398 - learning_rate: 1.9531e-06

Epoch 97/100
20/20 0s 3ms/step - accuracy: 0.5858 - loss: 0.9847 - val_accuracy: 0.6250 - val_loss: 0.9397 - learning_rate: 1.9531e-06

Epoch 98/100
20/20 0s 3ms/step - accuracy: 0.5760 - loss: 0.9766 - val_accuracy: 0.6250 - val_loss: 0.9401 - learning_rate: 1.9531e-06

Epoch 99/100
20/20 0s 4ms/step - accuracy: 0.5469 - loss: 1.0465 - val_accuracy: 0.6250 - val_loss: 0.9397 - learning_rate: 1.9531e-06

Epoch 100/100
20/20 0s 3ms/step - accuracy: 0.5592 - loss: 1.0095 - val_accuracy: 0.6250 - val_loss: 0.9394 - learning_rate: 9.7656e-07



5.3 Regression with Custom Training Loop

This code implements a regression model using a custom training loop in TensorFlow.

1. Data Generation and Preprocessing:

- A regression dataset is generated with 1000 samples and 10 features using `sklearn.datasets.make_regression`.
- The features (`X_reg`) and target values (`y_reg`) are standardized using `StandardScaler` to improve model training.

2. Custom Model Definition:

- A custom Keras model is defined with three dense layers:
 - The first layer has 16 units and uses ReLU activation.
 - The second layer has 8 units and also uses ReLU activation.
 - The third layer outputs a single value for regression.
- Dropout layers are added after the first two dense layers to help prevent overfitting.

3. Dataset Preparation:

- The dataset is converted to a TensorFlow `tf.data.Dataset` for efficient batching and shuffling.
- The dataset is split into batches of 32 samples.

4. Model and Optimizer Initialization:

- The custom regression model is initialized.
- The Adam optimizer is used with a learning rate of 0.001.

5. Training Loop with Early Stopping:

- The model is trained using a custom training loop. For each batch:
 - A forward pass is made through the model to get predictions.
 - The Mean Squared Error (MSE) loss is computed.
 - Gradients are calculated and applied to update the model weights.
- Early stopping is implemented to stop training if the loss doesn't improve for 10 consecutive epochs.

6. Prediction:

- After training, predictions are made on the entire dataset.

7. Plotting Results:

- The training loss over time is plotted.
- A scatter plot of predicted vs actual values is shown with a red line indicating perfect predictions.

8. Model Evaluation:

- The R^2 score is calculated and printed, which indicates how well the model fits the data.

```
In [70]: # Cell 3: Regression with Custom Training Loop
        """
        Implementation of regression with custom training loop and batch processing
        Dataset: Generated regression dataset
        """

        import numpy as np
        import matplotlib.pyplot as plt
        import tensorflow as tf
        from tensorflow import keras
        from sklearn.datasets import make_regression
        from sklearn.preprocessing import StandardScaler
        from sklearn.model_selection import train_test_split

        # Generate regression data
        X_reg, y_reg = make_regression(
            n_samples=1000,
            n_features=10,
            noise=0.1,
            random_state=42
        )

        # Standardize data
        scaler_X = StandardScaler()
        scaler_y = StandardScaler()
        X_reg_scaled = scaler_X.fit_transform(X_reg)
        y_reg_scaled = scaler_y.fit_transform(y_reg.reshape(-1, 1)).flatten()

        # Custom training loop with batch processing
        class CustomRegressionModel(keras.Model):
            def __init__(self):
                super(CustomRegressionModel, self).__init__()
                self.dense1 = keras.layers.Dense(16, activation='relu')
                self.dropout1 = keras.layers.Dropout(0.2)
                self.dense2 = keras.layers.Dense(8, activation='relu')
                self.dropout2 = keras.layers.Dropout(0.1)
                self.dense3 = keras.layers.Dense(1)

            def call(self, inputs, training=False):
                x = self.dense1(inputs)
                if training:
                    x = self.dropout1(x)
                x = self.dense2(x)
                if training:
                    x = self.dropout2(x)
                return self.dense3(x)

        # Create dataset
        train_dataset = tf.data.Dataset.from_tensor_slices((X_reg_scaled, y_reg_scaled))
        train_dataset = train_dataset.shuffle(buffer_size=1000).batch(32)

        # Initialize model and optimizer
        reg_model = CustomRegressionModel()
        optimizer = keras.optimizers.Adam(learning_rate=0.001)

        # Training loop with early stopping
        train_losses = []
        best_loss = float('inf')
        patience = 10
        patience_counter = 0

        # Define MSE loss function
        mse = tf.keras.losses.MeanSquaredError()

        @tf.function
        def train_step(x, y):
            with tf.GradientTape() as tape:
                predictions = reg_model(x, training=True)
                loss = mse(y, tf.squeeze(predictions))
            gradients = tape.gradient(loss, reg_model.trainable_variables)
            optimizer.apply_gradients(zip(gradients, reg_model.trainable_variables))
            return loss

        # Training with early stopping
        epochs = 100
        for epoch in range(epochs):
            epoch_loss = []
            for step, (x_batch, y_batch) in enumerate(train_dataset):
                loss = train_step(x_batch, y_batch)
                epoch_loss.append(float(loss))

            avg_loss = np.mean(epoch_loss)
```

```

train_losses.append(avg_loss)

if (epoch + 1) % 10 == 0:
    print(f'Epoch {epoch + 1}, Loss: {avg_loss:.4f}')

# Early stopping check
if avg_loss < best_loss:
    best_loss = avg_loss
    patience_counter = 0
else:
    patience_counter += 1
    if patience_counter >= patience:
        print(f'Early stopping triggered at epoch {epoch + 1}')
        break

# Make predictions
@tf.function
def predict(x):
    return reg_model(x, training=False)

y_pred = np.concatenate([predict(x_batch) for x_batch in tf.data.Dataset.from_tensor_slices(X_reg_scaled).batch(
    32)

# Plot regression training results
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(train_losses)
plt.title('Training Loss Over Time')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.scatter(y_reg_scaled, y_pred, alpha=0.5)
plt.plot([y_reg_scaled.min(), y_reg_scaled.max()],
         [y_reg_scaled.min(), y_reg_scaled.max()],
         'r--', lw=2)
plt.title('Predictions vs Actual Values')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.grid(True)

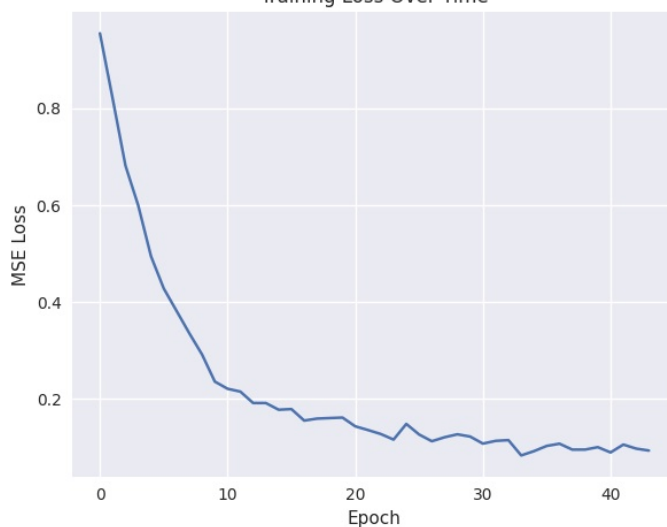
plt.tight_layout()
plt.show()

# Calculate and print final R² score
r2 = 1 - np.sum((y_reg_scaled - y_pred.flatten())**2) / np.sum((y_reg_scaled - np.mean(y_reg_scaled))**2)
print(f'\nFinal R² Score: {r2:.4f}')

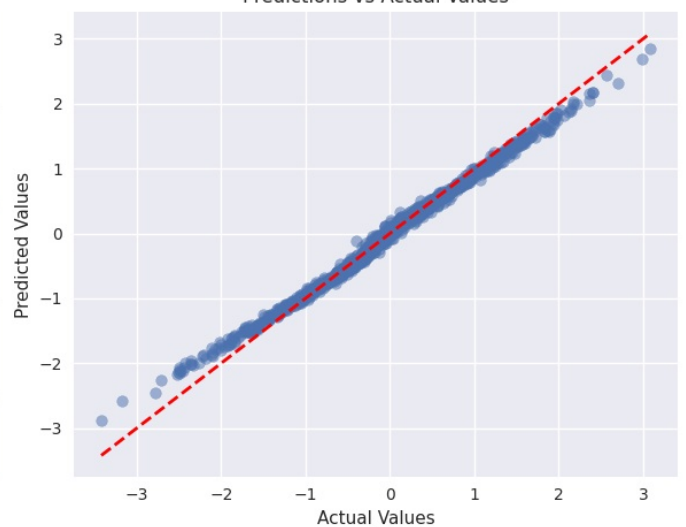
```

Epoch 10, Loss: 0.2361
 Epoch 20, Loss: 0.1618
 Epoch 30, Loss: 0.1226
 Epoch 40, Loss: 0.1007
 Early stopping triggered at epoch 44

Training Loss Over Time



Predictions vs Actual Values



Final R² Score: 0.9870